INFORME TÉCNICO DE ENTREGABLE — TALLER 2DO CORTE

Electiva Profesional I — Móviles con Flutter

ESTUDIANTE:
Brahian Andres Tenorio
FECHA: Octubre 2026

Repositorio Público GitHub:	https://github.com/brahiantenorio/-taller_datos_abiertos
Ramas Implementadas:	**main** (producción estable) **dev** (integración de desarrollo) **feature/taller_api_colombia** (rama del taller)
API Base Oficial:	https://api-colombia.com/api/v1 (Swagger: /swagger/index.html)

1. Resumen Ejecutivo del Proyecto

El presente taller tuvo como objetivo diseñar y construir una aplicación móvil completa y escalable en Flutter que consume **4 endpoints oficiales de la API Pública de Colombia**. La aplicación implementa una separación por capas (Config, Models, Services, Routes, Themes, Views y Widgets), navegación declarativa avanzada mediante **go_router** con paso de parámetros tipados (patrón Maestro-Detalle), control estricto de estados asíncronos (cargando, éxito, error con reintento y estado vacío por filtrado) y buenas prácticas de código limpio auditadas con *flutter analyze* y pruebas automáticas con *flutter test*.

2. Endpoints Consumidos (Mínimo 4 Requeridos)

Endpoint	Rutas API Consumidas	Datos y Atributos Principales
Departamentos	`GET /Department` `GET /Department/{id}`	Nombre, descripción, capital oficial, número de municipios, población, superficie (km²) y prefijo telefónico.
Presidentes	`GET /President` `GET /President/{id}`	Nombre, apellidos, fotografía oficial, partido político, fechas de inicio y fin de período, y biografía completa.
Atracciones Turísticas	`GET /TouristicAttraction` `GET /TouristicAttraction/{id}`	Nombre del destino, descripción histórica, galería de imágenes, coordenadas (latitud/longitud) y municipio.
Platos Típicos	`GET /TypicalDish` `GET /TypicalDish/{id}`	Nombre gastronómico, ingredientes detallados, departamento de origen, descripción tradicional y fotografía.

3. Paquetes Obligatorios Implementados

Paquete	Versión	Propósito y Justificación Técnica
http	1.6.0	Consumo de peticiones REST GET, cabeceras Accept: application/json, validación de códigos de estado (200, 400, 404, 500), timeouts (15s) y decodificación UTF-8.
go_router	18.1.1	Enrutamiento declarativo centralizado con rutas con nombre, rutas anidadas, extracción de parámetros :id y paso de payloads 'extra' para transiciones fluidas.
flutter_dotenv	6.0.1	Carga desacoplada de variables de entorno (.env) como API_BASE_URL y APP_NAME, con respaldo automático en caso de falta de archivo.

4. Arquitectura y Estructura de Carpetas

Se organizó el código siguiendo la arquitectura estricta solicitada en el taller:

Capa / Directorio	Archivos Clave	Responsabilidad
lib/config/	`app_config.dart` `constants.dart`	Gestión de variables (.env), constantes de endpoints y definición de nombres de rutas.

lib/models/	`department_model.dart`
	`president_model.dart`
	`touristic_attraction_model.dart`
	`typical_dish_model.dart`
		Modelos con tipado estricto, serialización *fromJson()*, deserialización *toJson()* y getters utilitarios.

lib/services/	`api_service.dart`	Capa HTTP con *http.Client*, timeouts, *ApiException* personalizada y mapeo de excepciones de red.
lib/routes/	`app_router.dart`	Configuración centralizada de *GoRouter*, rutas hijas, parámetros de ruta y pantalla 404.
lib/themes/	`app_theme.dart`	Tema Material 3 con paleta institucional colombiana, estilización de Cards y Badges.

lib/views/	`dashboard_view.dart`
	`..._list_view.dart`
	`..._detail_view.dart`
		Pantallas principales: Dashboard introductorio, Listados con ListView.builder y Detalles completos.

lib/widgets/	`loading_widget.dart`
	`error_view_widget.dart`
	`search_bar_widget.dart`
	`stat_badge_widget.dart`
	`network_image_with_fallback.dart`
		Componentes reutilizables desacoplados para renderizado de estados, imágenes e indicadores.

5. Navegación Declarativa con go_router (Maestro-Detalle)

Ruta ('name')	Path	Parámetros Enviados	Pantalla Destino
`dashboard`	`/`	Ninguno	DashboardView
`departments`	`/departments`	Ninguno	DepartmentsListView
`department-detail`	`/departments/:id`	:id (int) + DepartmentModel (extra)	DepartmentDetailView
`presidents`	`/presidents`	Ninguno	PresidentsListView
`president-detail`	`/presidents/:id`	:id (int) + PresidentModel (extra)	PresidentDetailView
`attractions`	`/attractions`	Ninguno	AttractionsListView
`attraction-detail`	`/attractions/:id`	:id (int) + TouristicAttractionModel (extra)	AttractionDetailView
`dishes`	`/dishes`	Ninguno	DishesListView
`dish-detail`	`/dishes/:id`	:id (int) + TypicalDishModel (extra)	DishDetailView

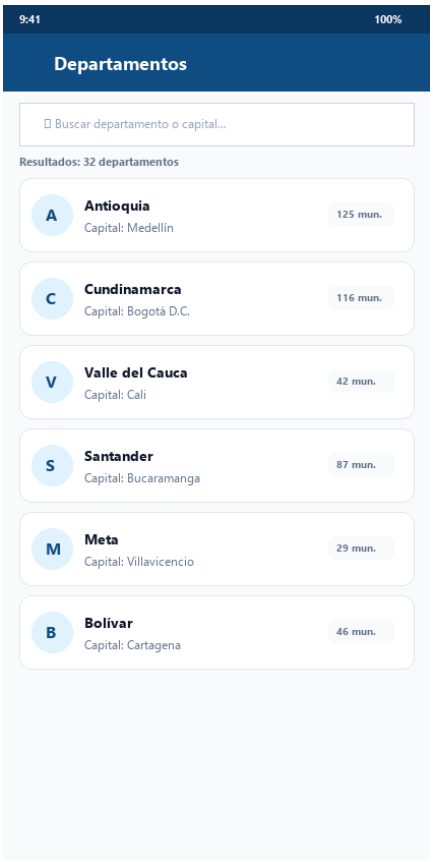
6. Manejo de Estados (Cargando / Éxito / Error)

• **Estado de Carga:** Indicador de progreso circular personalizado con mensajes descriptivos acordes a cada pantalla.
• **Estado de Éxito:** Renderizado de listas dinámicas con *ListView.builder*, tarjetas interactivas, barra de búsqueda en vivo y pull-to-refresh.
• **Estado de Error:** Interceptor de errores que traduce *SocketException*, *TimeoutException* y status codes (404, 500) en mensajes legibles con botón de reintento directo.

7. Evidencias Visuales de la Aplicación



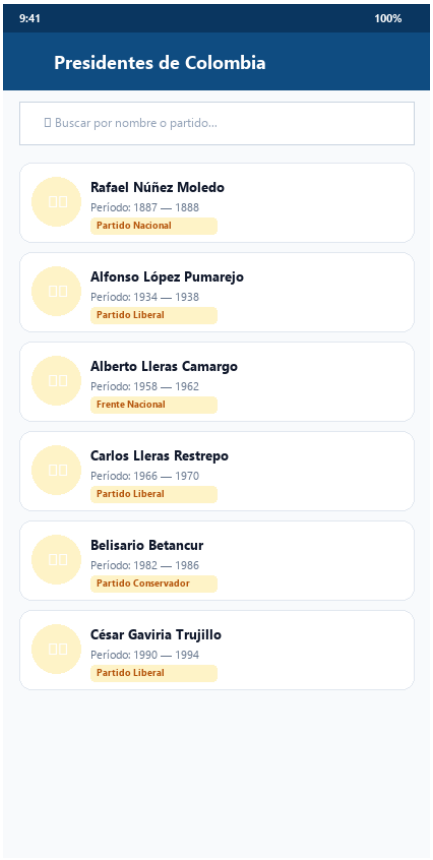
Dashboard Principal con Cards



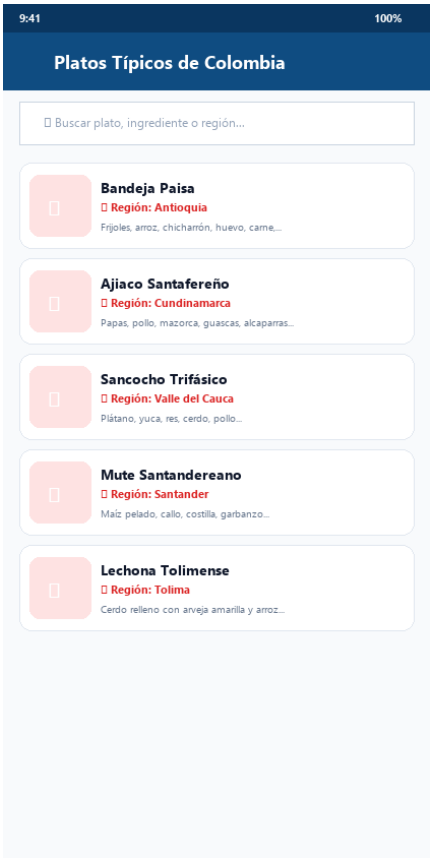
Listado con Buscador y Filtro



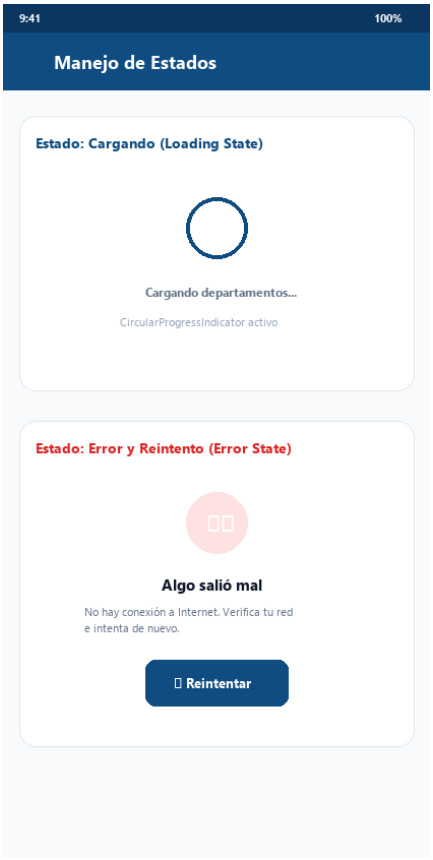
Detalle de Departamento (Métricas)



Listado Histórico de Presidentes



Gastronomía y Platos Típicos



Manejo de Estados (Carga y Error)

8. Flujo GitFlow y Pruebas Automatizadas

• **Flujo de Trabajo GitFlow:** Repositorio público con ramas **main** (estable), **dev** (integración) y **feature/taller_api_colombia** donde se registraron commits atómicos y semánticos (*feat:*, *chore:*, *refactor:*, *test:*, *docs:*).
• **Pull Request:** Integración desde *feature/taller_api_colombia* hacia *dev* y merge final a *main*.
• **Análisis Estático (flutter analyze):** 0 errores, 0 advertencias, 0 problemas de linter.
• **Pruebas Automatizadas (flutter test):** 10 pruebas unitarias y de widgets ejecutadas con 100% de éxito en models, api_service y widget tree.